E-Commerce Data Warehouse

AWS Glue / PySpark → S3 → Glue Data Catalog → Amazon Athena

Project Documentation

Prepared by: Adham

S3 Bucket: aws-ecommerce-data-adham-2026

Glue Database: adham_ecommerce_dw

Table of Contents

Table of Contents 2
1. Project Overview 4
1.1 Architecture 4
1.2 Why Athena instead of Redshift 6
2. Source Data 6
2.1 Source Tables, Keys, and Relationships 6
3. Data Quality Checks 7
3.1 Null Values 8
3.2 Duplicate Records 8
3.3 Invalid Values 8
4. Data Cleaning 9
5. Dimension Tables 10
6. Fact Tables 11
7. Incremental Order Ingestion (Lambda) 12
7.1 Checkpointing 12
7.2 Order Validation 12
7.3 Output 12
8. orders_json Processing (Glue Job) 12
9. Incremental Processing Design 13
10. S3 Processed Layer 14
11. Glue Crawler and Athena Setup 14
11.1 Crawler Configuration 15
11.2 Registered Tables 15
11.3 Column Naming Convention 15
12. Analytics Queries (Athena) 16
Task 29 — Top-line totals 16
Task 30 — Monthly sales 16
Task 31 — Sales by category 16
Task 31 — Sales by product 17
Task 31 — Sales by customer 17
Task 32 — Top 10 products by sales 17
Task 34 — Average order value 17
Task 35 — Order status analysis 18
Task 36 — Payment method analysis 18
Task 37 — Shipment performance 18

Task 38 — Running sales (window function) 18
Task 39 — Month-over-month growth 19
Task 40 — Top 3 products per category 19
Task 41 — Customer ranking 20
13. Issues Encountered and Resolutions 20
13.1 S3 Read Permission Error 20
13.2 Stale Tables in the Glue Data Catalog 21
13.3 Column-Naming Mismatch in Analytics SQL 21
14. Known Limitations and Follow-ups 21

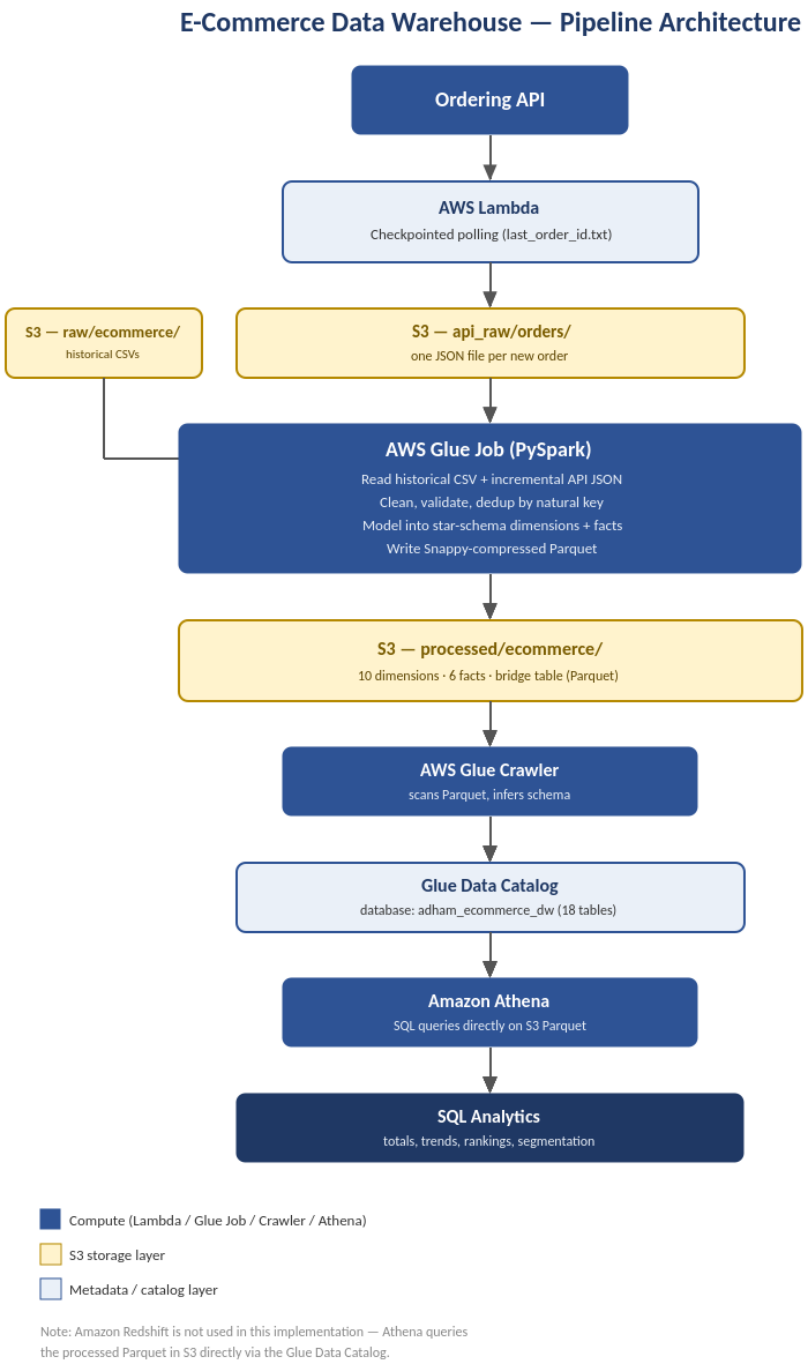
1. Project Overview

This project builds an e-commerce analytics data warehouse from a historical CSV source layer and a live ordering API. The original assignment specified Amazon Redshift as the final serving layer; this implementation instead uses an AWS Glue Crawler and Amazon Athena, querying Parquet files directly in S3 without loading them into a separate warehouse engine.

1.1 Architecture

The end-to-end pipeline has two ingestion paths that converge on the same transformation job:

- Historical CSVs, uploaded once to S3, covering customers, products, orders, and related reference data.
- A live ordering API, polled on a schedule by an AWS Lambda function, which checkpoints its progress and writes each new order as an individual JSON file to S3.
- Both sources are read, cleaned, and modeled together by a single AWS Glue (PySpark) job into a star-schema set of dimension and fact tables.
- An AWS Glue Crawler scans the transformed Parquet output and registers it as tables in the Glue Data Catalog.
- Amazon Athena queries those tables directly — no data is copied into Redshift or any other warehouse engine.



Pipeline flow (text form):

```
Ordering API
     |
     v
AWS Lambda (checkpointed polling)
     |
     v
S3  api_raw/orders/*.json  (one file per order)
     |
     v
AWS Glue Job (PySpark)
  - reads historical CSVs (raw/ecommerce/)
  - reads incremental API JSON (api_raw/orders/)
```

```
  - cleans, dedups, models into dims + facts
     |
     v
S3  processed/ecommerce/*  (Snappy Parquet)
     |
     v
AWS Glue Crawler  ->  Glue Data Catalog (adham_ecommerce_dw)
     |
     v
Amazon Athena  ->  SQL analytics
```

1.2 Why Athena instead of Redshift

Athena queries Parquet files in place using the schema registered by the Glue Crawler, so there is no separate load step, no cluster to provision, and no duplicate copy of the data to keep in sync. The trade-off is that all joins and aggregations run as on-demand distributed scans rather than against a tuned, indexed warehouse table — acceptable for this dataset's size and for periodic analytical queries rather than high-concurrency dashboards.

2. Source Data

Historical CSV data is stored at:

```
s3://aws-ecommerce-data-adham-2026/raw/ecommerce/
```

Incremental order data arrives as one JSON file per order at:

```
s3://aws-ecommerce-data-adham-2026/api_raw/orders/
```

2.1 Source Tables, Keys, and Relationships

Table	Primary Key	Foreign Keys
customers	CustomerID	—
categories	CategoryID	—
products	ProductID	CategoryID → categories
departments	DepartmentID	—
employees	EmployeeID	ManagerID → employees (self); DepartmentID → departments
suppliers	SupplierID	—
orders	OrderID	CustomerID → customers
order_details	OrderDetailID	OrderID → orders; ProductID → products
payments	PaymentID	OrderID → orders
product_suppliers	Composite (ProductID, SupplierID)	ProductID → products; SupplierID → suppliers
shippers	ShipperID	—
shipments	ShipmentID	OrderID → orders; ShipperID → shippers

Table	Primary Key	Foreign Keys
orders_json (API)	order.OrderID	order.CustomerID → customers; order_details[].ProductID → products; shipment.ShipperID → shippers (nullable)

Known source-data limitation:

There is no join path from products to departments in the source data — products carry CategoryID only, and categories carry no DepartmentID. dim_product includes a department_id column for schema completeness, but it is always NULL. This is a limitation of the source data, not a defect in the transformation logic.

```
root
 |-- CustomerID: integer (nullable = true)
 |-- FirstName: string (nullable = true)
 |-- LastName: string (nullable = true)
 |-- Email: string (nullable = true)
 |-- City: string (nullable = true)
 |-- Country: string (nullable = true)
 |-- RegistrationDate: timestamp (nullable = true)

root
 |-- CategoryID: integer (nullable = true)
 |-- CategoryName: string (nullable = true)

root
 |-- ProductID: integer (nullable = true)
 |-- CategoryID: integer (nullable = true)
 |-- ProductName: string (nullable = true)
 |-- Brand: string (nullable = true)
 |-- Price: double (nullable = true)
 |-- Cost: double (nullable = true)
 |-- Stock: integer (nullable = true)

root
 |-- DepartmentID: integer (nullable = true)
 |-- DepartmentName: string (nullable = true)

root
 |-- EmployeeID: integer (nullable = true)
 |-- ManagerID: integer (nullable = true)
 |-- DepartmentID: integer (nullable = true)
 |-- FirstName: string (nullable = true)
 |-- LastName: string (nullable = true)
 |-- Salary: double (nullable = true)
 |-- HireDate: timestamp (nullable = true)


root
 |-- SupplierID: integer (nullable = true)
 |-- SupplierName: string (nullable = true)
 |-- Country: string (nullable = true)

root
 |-- OrderID: integer (nullable = true)
 |-- CustomerID: integer (nullable = true)
 |-- OrderDate: timestamp (nullable = true)
 |-- Status: string (nullable = true)

root
 |-- OrderDetailID: integer (nullable = true)
 |-- OrderID: integer (nullable = true)
 |-- ProductID: integer (nullable = true)
 |-- Quantity: integer (nullable = true)
 |-- UnitPrice: double (nullable = true)
 |-- Discount: integer (nullable = true)

root
 |-- PaymentID: integer (nullable = true)
 |-- OrderID: integer (nullable = true)
 |-- PaymentMethod: string (nullable = true)
 |-- PaymentDate: timestamp (nullable = true)
 |-- Amount: double (nullable = true)

root
 |-- ProductID: integer (nullable = true)
 |-- SupplierID: integer (nullable = true)

root
 |-- ShipperID: integer (nullable = true)
 |-- CompanyName: string (nullable = true)


root
 |-- ShipmentID: integer (nullable = true)
 |-- OrderID: integer (nullable = true)
 |-- ShipperID: integer (nullable = true)
 |-- ShipDate: timestamp (nullable = true)
 |-- DeliveryDate: timestamp (nullable = true)
```

3. Data Quality Checks

Performed against the raw historical tables, before any cleaning or renaming.

3.1 Null Values

For every source table and column, null count and null percentage were computed and reported as (table, column, null_count, null_percentage).

```
+---------+---------+----------+---------------+
|table    |column   |null_count|null_percentage|
+---------+---------+----------+---------------+
|employees|ManagerID|1         |0.5            |
+---------+---------+----------+---------------+
```

3.2 Duplicate Records

Duplicates were checked per table against the business key listed in Section 2.1 (e.g. CustomerID for customers, OrderID for orders). Handling approach: rows sharing a business key are treated as duplicates and resolved with dropDuplicates() during the cleaning step, after the raw counts were reported for visibility.

```
customers (CustomerID): 0 duplicated key value(s)
products (ProductID): 0 duplicated key value(s)
categories (CategoryID): 0 duplicated key value(s)
departments (DepartmentID): 0 duplicated key value(s)
employees (EmployeeID): 0 duplicated key value(s)
suppliers (SupplierID): 0 duplicated key value(s)
orders (OrderID): 0 duplicated key value(s)
order_details (OrderDetailID): 0 duplicated key value(s)
payments (PaymentID): 0 duplicated key value(s)
shippers (ShipperID): 0 duplicated key value(s)
shipments (ShipmentID): 0 duplicated key value(s)
```

3.3 Invalid Values

The following checks were run against the raw data:

- Negative product prices
- Negative order-line quantities
- Negative payment amounts
- Invalid email address formats (regex validation)
- Null / invalid order dates
- Shipments with a delivery date earlier than their ship date
- Orders referencing a nonexistent customer (left-anti join against customers)
- Order details referencing a nonexistent product (left-anti join against products)
- Payments referencing a nonexistent order (left-anti join against orders)
- Shipments referencing a nonexistent order (left-anti join against orders)

```
--- Negative product prices ---
+---------+----------+-----------+-----+-----+----+-----+
|ProductID|CategoryID|ProductName|Brand|Price|Cost|Stock|
+---------+----------+-----------+-----+-----+----+-----+
+---------+----------+-----------+-----+-----+----+-----+

--- Negative quantities ---
+-------------+-------+---------+--------+---------+--------+
|OrderDetailID|OrderID|ProductID|Quantity|UnitPrice|Discount|
+-------------+-------+---------+--------+---------+--------+
+-------------+-------+---------+--------+---------+--------+

--- Negative payment amounts ---
+---------+-------+-------------+-----------+------+
|PaymentID|OrderID|PaymentMethod|PaymentDate|Amount|
+---------+-------+-------------+-----------+------+
+---------+-------+-------------+-----------+------+

--- Invalid email addresses ---
+----------+---------+--------+-----+----+-------+----------------+
|CustomerID|FirstName|LastName|Email|City|Country|RegistrationDate|
+----------+---------+--------+-----+----+-------+----------------+
+----------+---------+--------+-----+----+-------+----------------+

--- Invalid dates (orders with null OrderDate, since these came through as timestamp already parsed) ---
+-------+----------+---------+------+
|OrderID|CustomerID|OrderDate|Status|
+-------+----------+---------+------+
+-------+----------+---------+------+


 --- Shipments delivered before they were shipped ---
 +----------+-------+---------+--------+------------+
 |ShipmentID|OrderID|ShipperID|ShipDate|DeliveryDate|
 +----------+-------+---------+--------+------------+
 +----------+-------+---------+--------+------------+

 --- Orders referencing nonexistent customers ---
 +----------+-------+---------+------+
 |CustomerID|OrderID|OrderDate|Status|
 +----------+-------+---------+------+
 +----------+-------+---------+------+

 --- Order details referencing nonexistent products ---
 +---------+-------------+-------+--------+---------+--------+
 |ProductID|OrderDetailID|OrderID|Quantity|UnitPrice|Discount|
 +---------+-------------+-------+--------+---------+--------+
 +---------+-------------+-------+--------+---------+--------+

 --- Payments referencing nonexistent orders ---
 +-------+---------+-------------+-----------+------+
 |OrderID|PaymentID|PaymentMethod|PaymentDate|Amount|
 +-------+---------+-------------+-----------+------+
 +-------+---------+-------------+-----------+------+

 --- Shipments referencing nonexistent orders ---
 +-------+----------+---------+--------+------------+
 |OrderID|ShipmentID|ShipperID|ShipDate|DeliveryDate|
 +-------+----------+---------+--------+------------+
 +-------+----------+---------+--------+------------+
```

4. Data Cleaning

Applied uniformly across all historical tables before modeling:

- Column names standardized: trimmed, lower-cased, spaces and hyphens replaced with underscores.

- String values trimmed of leading/trailing whitespace; select fields (names, statuses, payment methods) upper-cased or title-cased for consistency.
- Email addresses lower-cased.
- Columns cast to their appropriate types (long, decimal(12,2), date, etc.), rather than left as Spark's inferred types.
- Negative numeric values (quantity, unit price, discount) floored to zero rather than dropped, to preserve the order line while removing an impossible value.
- Duplicate records removed via dropDuplicates() on each table's business key.
- Dates validated and parsed with to_date() / to_timestamp().

Cleaned output is written to:

```
s3://aws-ecommerce-data-adham-2026/processed/ecommerce/
```

5. Dimension Tables

Each dimension uses its natural source-system identifier as its key column (e.g. customer_id, product_id) rather than a separately generated surrogate key. This keeps the model simple and the joins direct; the trade-off, if the warehouse later needs to track slowly-changing history per row, is that a generated surrogate key would decouple the warehouse key from the source system's ID.

Dimension	Built from	Key attributes
dim_customer	customers	customer_id, first_name, last_name, full_name, email, city, country, registration_date
dim_category	categories	category_id, category_name
dim_department	departments	department_id, department_name
dim_product	products, categories	product_id, category_id, product_name, brand, price, cost, stock, department_id (always NULL), margin, margin_percent
dim_supplier	suppliers	supplier_id, supplier_name, country
dim_employee	employees	employee_id, manager_id, department_id, first_name, last_name, full_name, salary, hire_date
dim_shipper	shippers	shipper_id, company_name
dim_payment_method	payments (distinct)	payment_method_id, payment_method
dim_order_status	orders (distinct)	order_status_id, order_status
dim_date	generated calendar range	date, date_id, year, quarter, month, month_name, day, day_of_week, day_name, week_of_year, is_weekend

Dim_customer:

#	customer_id	first_name	last_name	full_name	email	city	country	registration_date
1	3	Jill	Rhodes	Jill Rhodes	jill.rhodes3@example.com	Giza	United Kingdom	2020-12-22
2	4	Patricia	Miller	Patricia Miller	patricia.miller4@example.com	London	Germany	2020-05-10
3	6	Jeffery	Wagner	Jeffery Wagner	jeffery.wagner6@example.com	Dubai	United Kingdom	2020-04-18
4	9	Jeffrey	Lawrence	Jeffrey Lawrence	jeffrey.lawrence9@example.com	London	Kuwait	2020-01-27
5	11	Linda	Wolfe	Linda Wolfe	linda.wolfe11@example.com	Abu Dhabi	UAE	2022-05-31

Dim_product:

#	product_id	category_id	product_name	brand	price	cost	stock	department_id	margin	margin_percent
1	1	12	Bose Printer 1	Bose	261.16	140.65	399		120.51	46.14
2	2	3	Lenovo Mouse 2	Lenovo	1874.20	1460.92	155		413.28	22.05
3	3	12	Sony Headphones 3	Sony	1272.54	846.77	315		425.77	33.46
4	4	11	LG Smart Watch 4	LG	1180.67	637.81	95		542.86	45.98
5	5	16	Bose Television 5	Bose	253.18	165.51	375		87.67	34.63

6. Fact Tables

Each fact table's grain is stated explicitly below, since it determines what a single row means and what can safely be summed.

Fact table	Grain	Built from	Key columns
fact_order	One row per order	orders	order_id, customer_id, date_id, order_date, order_status_id
fact_order_detail	One row per order line item	orders, order_details, products	order_detail_id, order_id, product_id, quantity, unit_price, discount, gross_amount, discount_amount, total_amount
fact_payment	One row per payment transaction	payments	payment_id, order_id, date_id, payment_date, payment_method_id, amount
fact_shipment	One row per shipment	shipments, orders, shippers	shipment_id, order_id, shipper_id, ship_date, delivery_date, shipping_days
fact_customer_sales	One row per customer, per day	fact_order_detail, fact_order (aggregated)	customer_id, date_id, order_count, quantity, sales, profit
fact_product_sales	One row per product, per day	fact_order_detail, fact_order, dim_product (aggregated)	product_id, date_id, order_count, quantity_sold, gross_sales, discount, net_sales, cost, profit

fact_customer_sales and fact_product_sales are aggregated facts derived from fact_order_detail joined back to fact_order for its date_id, then grouped to the required daily grain — this was verified by confirming row count equals the count of distinct (key, date_id) pairs for each table before writing to Parquet.

```
fact_customer_sales grain check — should equal distinct (customer_id, date_id) pairs:
43116 43116
fact_product_sales grain check — should equal distinct (product_id, date_id) pairs:
95311 95311
```

7. Incremental Order Ingestion (Lambda)

New orders are pulled from the ordering API by a scheduled AWS Lambda function rather than a bulk reload. The function is checkpointed, so each invocation only fetches orders created after the last one it successfully processed.

7.1 Checkpointing

- The last successfully processed OrderID is stored as a plain text file at api_raw/checkpoint/last_order_id.txt.
- If no checkpoint exists yet, ingestion starts from a fixed INITIAL_ORDER_ID (2,000,000).
- On each run, the function requests orders starting from (last_order_id + 1), in batches of up to 1,000.
- Only orders with an OrderID greater than the checkpoint are uploaded; anything at or below it is skipped as already processed.
- The checkpoint is advanced to the highest OrderID actually uploaded in that run, and only after all uploads in the batch succeed.

7.2 Order Validation

Before upload, each order event is validated: the order object and its OrderID must be present, and order_details must be a list. Missing payment, shipment, or summary blocks are logged as warnings rather than treated as fatal, since an order can legitimately have no shipment yet.

7.3 Output

Each new order is written as its own JSON object to:

```
s3://aws-ecommerce-data-adham-2026/api_raw/orders/order_<OrderID>.json
```

This one-file-per-order layout, combined with the checkpoint, is what makes the ingestion naturally idempotent: re-running the Lambda after a partial failure re-fetches from the API but will not re-upload or duplicate any order whose ID is at or below the saved checkpoint.

8. orders_json Processing (Glue Job)

The Glue job reads every JSON file under api_raw/orders/ with Spark's multiline JSON reader, then:

- Parses the nested order, order_details[], payment, shipment, and summary structures using Spark's dot-path column access.
- Explodes order_details[] into one row per order line item.
- Casts every field to its target type (long IDs, decimal(12,2) money fields, parsed dates).
- Standardizes status and payment-method text to upper case, matching the historical data's convention.
- Unions the parsed API orders, order details, payments, and shipments with their historical-CSV counterparts using unionByName, then removes duplicates with dropDuplicates() on each table's natural key — this is what prevents an order from appearing twice if it exists in both the historical load and a later API pull.

```
API JSON schema:
root
 |-- metadata: struct (nullable = true)
 |    |-- event_id: string (nullable = true)
 |    |-- event_timestamp: string (nullable = true)
 |    |-- event_type: string (nullable = true)
 |    |-- source: string (nullable = true)
 |-- order: struct (nullable = true)
 |    |-- CustomerID: long (nullable = true)
 |    |-- OrderDate: string (nullable = true)
 |    |-- OrderID: long (nullable = true)
 |    |-- Status: string (nullable = true)
 |-- order_details: array (nullable = true)
 |    |-- element: struct (containsNull = true)
 |    |    |-- Discount: double (nullable = true)
 |    |    |-- OrderDetailID: long (nullable = true)
 |    |    |-- OrderID: long (nullable = true)
 |    |    |-- ProductID: long (nullable = true)
 |    |    |-- Quantity: long (nullable = true)
 |    |    |-- UnitPrice: double (nullable = true)
 |-- payment: struct (nullable = true)
 |    |-- Amount: double (nullable = true)
 |    |-- OrderID: long (nullable = true)
 |    |-- PaymentDate: string (nullable = true)
 |    |-- PaymentID: long (nullable = true)
 |    |-- PaymentMethod: string (nullable = true)
 |    |-- PaymentStatus: string (nullable = true)
 |-- shipment: struct (nullable = true)
 |    |-- DeliveryDate: string (nullable = true)
 |    |-- OrderID: long (nullable = true)
 |    |-- ShipDate: string (nullable = true)
 |    |-- ShipmentID: long (nullable = true)
 |    |-- ShipperID: long (nullable = true)
 |-- summary: struct (nullable = true)
 |    |-- discount: double (nullable = true)
 |    |-- shipping_cost: double (nullable = true)
 |    |-- subtotal: double (nullable = true)
 |    |-- tax_amount: double (nullable = true)
 |    |-- tax_rate: double (nullable = true)
 |    |-- total_amount: double (nullable = true)
```

9. Incremental Processing Design

The pipeline is designed around two complementary incremental mechanisms, one at ingestion and one at transformation:

- Ingestion (Lambda): the OrderID checkpoint means each Lambda run only pulls orders newer than the last one it processed — the API itself is never asked to resend history.
- Transformation (Glue job): dropDuplicates() on each table's natural key after unioning historical and API-sourced records means that even if the same order file is read twice (e.g. after a Glue job retry), it collapses to a single row rather than double-counting in the fact tables.

Current limitation: the Glue job itself re-reads all historical CSVs and the entire api_raw/orders/ prefix on every run, then overwrites the processed Parquet output. This is correct, but not minimal — a stricter incremental design would track which API JSON files have already been processed (e.g. a manifest or Glue job bookmark) and merge only new records into the existing Parquet output instead of recomputing every dimension and fact from scratch each run.

10. S3 Processed Layer

All dimensions and facts are written as Snappy-compressed Parquet under:

```
s3://aws-ecommerce-data-adham-2026/processed/ecommerce/
```

Directory structure:

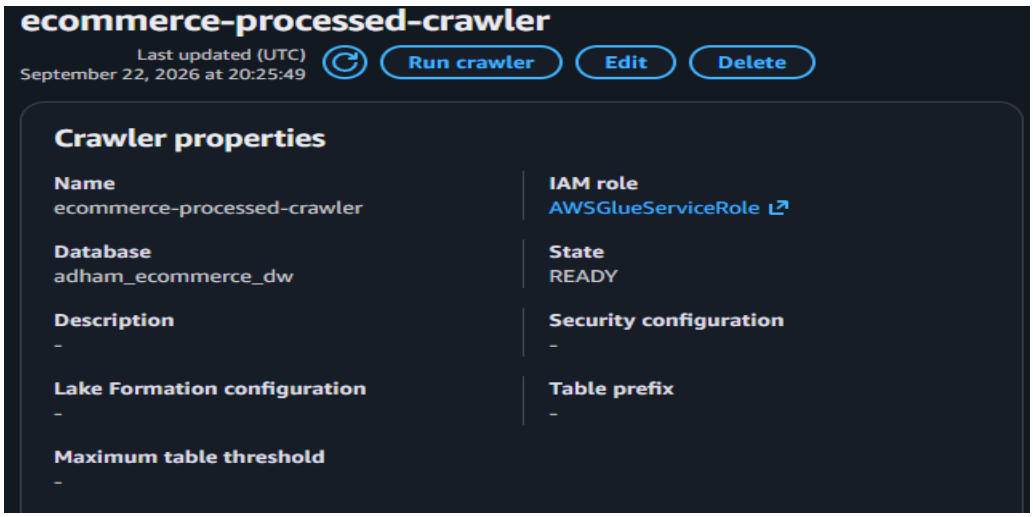
```
processed/ecommerce/
├── dim_customer/
├── dim_product/
├── dim_category/
├── dim_department/
├── dim_supplier/
├── dim_employee/
├── dim_shipper/
├── dim_date/
├── dim_payment_method/
├── dim_order_status/
├── fact_order/
├── fact_order_detail/
├── fact_payment/
├── fact_shipment/
├── fact_customer_sales/
├── fact_product_sales/
├── bridge_product_supplier/
└── api_orders/          (partitioned by year, month)
```

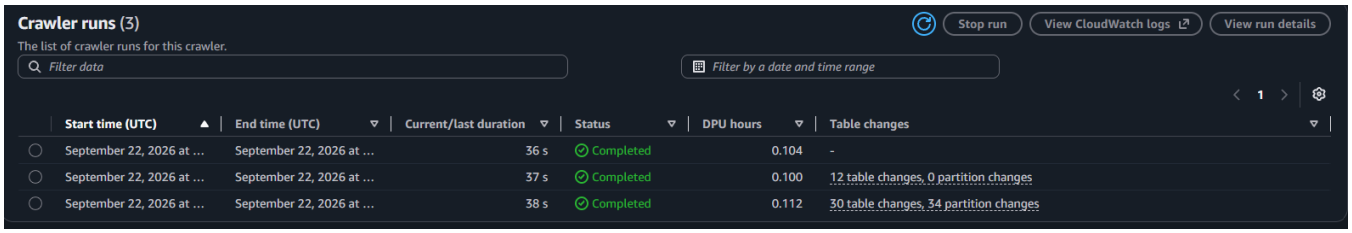


11. Glue Crawler and Athena Setup

11.1 Crawler Configuration

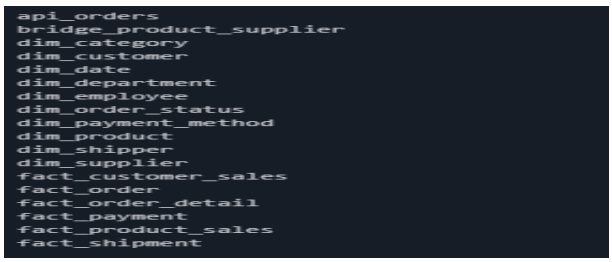
Setting	Value
Data source	S3
S3 path	s3://aws-ecommerce-data-adham-2026/processed/ecommerce/
Target database	adham_ecommerce_dw
Format	Parquet
IAM role	AWSGlueServiceRole (extended — see Section 13.1)
Schedule	On demand





11.2 Registered Tables

After cleanup (see Section 13.2), the crawler registers exactly 18 tables in adham_ecommerce_dw: 10 dimensions, 6 facts, the supplier bridge table, and the raw api_orders table.



11.3 Column Naming Convention

Dimension and fact tables use their source system's natural key column names (customer_id, product_id, date_id, order_status_id, payment_method_id, category_id) as their join keys, and dim_date's calendar column is named date rather than full_date. All SQL in Section 12 is written against these actual column names.

12. Analytics Queries (Athena)

All queries below were run and validated in Amazon Athena against the adham_ecommerce_dw database. Note that Task 33 (top 10 customers by spending) is covered by the Task 41 ranking query, and Task 31's three breakdowns (category, product, customer) are each included separately since department-level rollup has no join path (Section 2.1).

Task 29 — Top-line totals

```
SELECT
    ROUND(SUM(fod.total_amount), 2) AS total_sales,
    COUNT(DISTINCT fo.order_id) AS total_orders,
    COUNT(DISTINCT fo.customer_id) AS total_customers,
    COUNT(DISTINCT fod.product_id) AS total_products,
    SUM(fod.quantity) AS total_quantity_sold,
    ROUND(SUM(fod.total_amount - fod.discount_amount), 2) AS total_profit
FROM adham_ecommerce_dw.fact_order_detail fod
JOIN adham_ecommerce_dw.fact_order fo ON fod.order_id = fo.order_id;
```

#	total_sales	total_orders	total_customers	total_products	total_quantity_sold	total_profit
1	781299113.48	43216	9939	1187	550062	721806208.55

Task 30 — Monthly sales

```
SELECT
    d.year, d.month, d.month_name,
    ROUND(SUM(fod.total_amount), 2) AS sales,
    COUNT(DISTINCT fo.order_id) AS orders,
    SUM(fod.quantity) AS quantity,
    ROUND(SUM(fod.total_amount - fod.discount_amount), 2) AS profit
FROM adham_ecommerce_dw.fact_order_detail fod
JOIN adham_ecommerce_dw.fact_order fo ON fod.order_id = fo.order_id
JOIN adham_ecommerce_dw.dim_date d ON fo.date_id = d.date_id
GROUP BY d.year, d.month, d.month_name
ORDER BY d.year, d.month;
```

#	year	month	month_name	sales	orders	quantity	profit
1	2024	1	January	24204411.52	1371	17214	22396683.57
2	2024	2	February	22439615.72	1242	15850	20694812.55
3	2024	3	March	24653244.12	1386	17526	22755802.91
4	2024	4	April	24577733.29	1362	17056	22733650.53
5	2024	5	May	24800238.43	1394	17461	22964181.81

Task 31 — Sales by category

```
SELECT dc.category_name, ROUND(SUM(fod.total_amount), 2) AS sales
FROM adham_ecommerce_dw.fact_order_detail fod
JOIN adham_ecommerce_dw.dim_product dp ON fod.product_id = dp.product_id
JOIN adham_ecommerce_dw.dim_category dc ON dp.category_id = dc.category_id
GROUP BY dc.category_name
ORDER BY sales DESC;
```

#	category_name	sales
1	Gaming	53200334.76
2	Electronics	52504073.57
3	Clothing	48755269.70
4	Jewelry	45858910.70
5	Furniture	45447445.00

Task 31 — Sales by product

```
SELECT dp.product_name, ROUND(SUM(fod.total_amount), 2) AS sales
FROM adham_ecommerce_dw.fact_order_detail fod
JOIN adham_ecommerce_dw.dim_product dp ON fod.product_id = dp.product_id
GROUP BY dp.product_name
ORDER BY sales DESC;
```

#	product_name	sales
1	Microsoft Printer 582	1973379.08
2	Canon Printer 794	1903432.08
3	HP Laptop 650	1833759.72
4	Dell Keyboard 71	1752721.19
5	Adidas Gaming Console 911	1729415.16

Task 31 — Sales by customer

```
SELECT dc.full_name, ROUND(SUM(fod.total_amount), 2) AS sales
FROM adham_ecommerce_dw.fact_order_detail fod
JOIN adham_ecommerce_dw.fact_order fo ON fod.order_id = fo.order_id
JOIN adham_ecommerce_dw.dim_customer dc ON fo.customer_id = dc.customer_id
GROUP BY dc.full_name
ORDER BY sales DESC;
```

#	full_name	sales
1	David Harris	779808.05
2	John Williams	485476.58
3	John Reed	481450.61
4	Jeffrey Ramirez	473461.80
5	John Grant	468781.17

Task 32 — Top 10 products by sales

```
SELECT dp.product_name, ROUND(SUM(fod.total_amount), 2) AS sales
FROM adham_ecommerce_dw.fact_order_detail fod
JOIN adham_ecommerce_dw.dim_product dp ON fod.product_id = dp.product_id
GROUP BY dp.product_name
ORDER BY sales DESC
LIMIT 10;
```

#	product_name	sales
1	Microsoft Printer 582	1973379.08
2	Canon Printer 794	1903432.08
3	HP Laptop 650	1833759.72
4	Dell Keyboard 71	1752721.19
5	Adidas Gaming Console 911	1729415.16

Task 34 — Average order value

```
SELECT ROUND(SUM(total_amount) / COUNT(DISTINCT order_id), 2) AS average_order_value
FROM adham_ecommerce_dw.fact_order_detail;
```

#	average_order_value
1	18078.93

Task 35 — Order status analysis

```
SELECT dos.order_status, COUNT(DISTINCT fo.order_id) AS order_count,
    ROUND(SUM(fod.total_amount), 2) AS total_value
FROM adham_ecommerce_dw.fact_order fo
JOIN adham_ecommerce_dw.fact_order_detail fod ON fo.order_id = fod.order_id
JOIN adham_ecommerce_dw.dim_order_status dos ON fo.order_status_id = dos.order_status_id
GROUP BY dos.order_status
ORDER BY order_count DESC;
```

#	order_status	order_count	total_value
1	DELIVERED	8693	157367032.79
2	SHIPPED	8633	154682864.18
3	PENDING	8633	156637807.87
4	CANCELLED	8590	155574558.29
5	PROCESSING	8589	156781910.80
6	CONFIRMED	78	254939.55

Task 36 — Payment method analysis

```
SELECT dpm.payment_method, COUNT(*) AS number_of_transactions,
    ROUND(SUM(fp.amount), 2) AS total_payment_amount,
    ROUND(AVG(fp.amount), 2) AS average_payment_amount
FROM adham_ecommerce_dw.fact_payment fp
JOIN adham_ecommerce_dw.dim_payment_method dpm ON fp.payment_method_id = dpm.payment_method_id
GROUP BY dpm.payment_method
ORDER BY total_payment_amount DESC;
```

#	payment_method	number_of_transactions	total_payment_amount	average_payment_amount
1	CREDIT CARD	9112	23306450.93	2557.78
2	CASH	9082	23078562.34	2541.13
3	BANK TRANSFER	9038	22646345.38	2505.68
4	PAYPAL	8937	22209529.14	2485.12
5	DEBIT CARD	8894	22076406.90	2482.17
6	CASH ON DELIVERY	17	82024.67	4824.98

Task 37 — Shipment performance

```
SELECT
    COUNT(*) AS total_shipments,
    ROUND(AVG(shipping_days), 2) AS average_delivery_days,
    MIN(shipping_days) AS minimum_delivery_days,
    MAX(shipping_days) AS maximum_delivery_days
FROM adham_ecommerce_dw.fact_shipment
WHERE shipping_days IS NOT NULL;
```

#	total_shipments	average_delivery_days	minimum_delivery_days	maximum_delivery_days
1	40078	5.49	1	10

Task 38 — Running sales (window function)

```
SELECT d.date AS order_date, ROUND(SUM(fod.total_amount), 2) AS daily_sales,
    ROUND(SUM(SUM(fod.total_amount)) OVER (ORDER BY d.date), 2) AS running_sales
FROM adham_ecommerce_dw.fact_order_detail fod
JOIN adham_ecommerce_dw.fact_order fo ON fod.order_id = fo.order_id
JOIN adham_ecommerce_dw.dim_date d ON fo.date_id = d.date_id
```

```
GROUP BY d.date
ORDER BY d.date;
```

#	order_date	daily_sales	running_sales
1	2024-01-01	734262.81	734262.81
2	2024-01-02	520023.83	1254286.64
3	2024-01-03	893713.86	2148000.50
4	2024-01-04	536710.99	2684711.49
5	2024-01-05	1078249.47	3762960.96

Task 39 — Month-over-month growth

```
WITH monthly AS (
    SELECT d.year, d.month, ROUND(SUM(fod.total_amount), 2) AS current_month_sales
    FROM adham_ecommerce_dw.fact_order_detail fod
    JOIN adham_ecommerce_dw.fact_order fo ON fod.order_id = fo.order_id
    JOIN adham_ecommerce_dw.dim_date d ON fo.date_id = d.date_id
    GROUP BY d.year, d.month
),
with_previous AS (
    SELECT year, month, current_month_sales,
        LAG(current_month_sales) OVER (ORDER BY year, month) AS previous_month_sales
    FROM monthly
)
SELECT year, month, current_month_sales, previous_month_sales,
    ROUND(current_month_sales - previous_month_sales, 2) AS sales_difference,
    ROUND(100.0 * (current_month_sales - previous_month_sales)
        / NULLIF(previous_month_sales, 0), 2) AS growth_percentage
FROM with_previous
ORDER BY year, month;
```

#	year	month	current_month_sales	previous_month_sales	sales_difference	growth_percentage
1	2024	1	24204411.52			
2	2024	2	22439615.72	24204411.52	-1764795.80	-7.29
3	2024	3	24653244.12	22439615.72	2213628.40	9.86
4	2024	4	24577733.29	24653244.12	-75510.83	-0.31
5	2024	5	24800238.43	24577733.29	222505.14	0.91

Task 40 — Top 3 products per category

```
WITH product_sales AS (
    SELECT dc.category_name, dp.product_name, SUM(fod.total_amount) AS sales
    FROM adham_ecommerce_dw.fact_order_detail fod
    JOIN adham_ecommerce_dw.dim_product dp ON fod.product_id = dp.product_id
    JOIN adham_ecommerce_dw.dim_category dc ON dp.category_id = dc.category_id
    GROUP BY dc.category_name, dp.product_name
),
ranked AS (
    SELECT category_name, product_name, ROUND(sales, 2) AS sales,
        ROW_NUMBER() OVER (PARTITION BY category_name ORDER BY sales DESC) AS product_rank
    FROM product_sales
)
SELECT * FROM ranked WHERE product_rank <= 3 ORDER BY category_name, product_rank;
```

#	category_name	product_name	sales	product_rank
1	Accessories	Samsung Smartphone 212	1704521.63	1
2	Accessories	Bose Headphones 967	1584324.56	2
3	Accessories	LG Headphones 13	1453617.31	3
4	Automotive	Sony Chair 335	1700456.24	1
5	Automotive	Microsoft Smartphone 441	1488569.90	2

Task 41 — Customer ranking

```
WITH customer_spending AS (
    SELECT dc.full_name, SUM(fod.total_amount) AS total_spending,
        COUNT(DISTINCT fo.order_id) AS order_count
    FROM adham_ecommerce_dw.fact_order_detail fod
    JOIN adham_ecommerce_dw.fact_order fo ON fod.order_id = fo.order_id
    JOIN adham_ecommerce_dw.dim_customer dc ON fo.customer_id = dc.customer_id
    GROUP BY dc.full_name
)
SELECT full_name, ROUND(total_spending, 2) AS total_spending, order_count,
    RANK() OVER (ORDER BY total_spending DESC) AS customer_rank
FROM customer_spending
ORDER BY customer_rank;
```

#	full_name	total_spending	order_count	customer_rank
1	David Harris	779808.05	34	1
2	John Williams	485476.58	28	2
3	John Reed	481450.61	23	3
4	Jeffrey Ramirez	473461.80	17	4
5	John Grant	468781.17	16	5

13. Issues Encountered and Resolutions

Three issues came up while standing up the crawler/Athena layer, documented here for traceability.

13.1 S3 Read Permission Error

Symptom: spark.read.csv() failed with AnalysisException: Path does not exist, even though the file was confirmed present via boto3's list_objects_v2 and Spark's own filesystem listStatus() could see the parent directory.

Root cause: the Glue job runs as the AWSGlueServiceRole, whose default AWS-managed policy does not grant access to arbitrary S3 buckets. A direct boto3 head_object() call confirmed a 403 Forbidden on the object itself — ListBucket was allowed, but GetObject was not.

Fix: added an inline policy to the role granting GetObject, PutObject, ListBucket, and DeleteObject on both the bucket ARN and the bucket/* ARN:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject","s3:PutObject","s3:ListBucket","s3:DeleteObject"],
    "Resource": [
      "arn:aws:s3:::aws-ecommerce-data-adham-2026",
      "arn:aws:s3:::aws-ecommerce-data-adham-2026/*"
    ]
```

```
  }]
}
```

13.2 Stale Tables in the Glue Data Catalog

Symptom: the first crawler run registered 30 tables instead of the expected 18. The extra tables (categories, customers, orders, order_details, payments, products, shipments, plus category_sales, customer_sales, fact_sales, monthly_sales, product_sales) were leftovers from an earlier draft of the transformation job that had written to the same processed/ecommerce/ prefix under different naming, before the dim_/fact_ convention was finalized.

Verified via S3 object timestamps (older than the current run) before removing anything, since the bucket's naming made it worth confirming the data was this project's own earlier output rather than an unrelated project.

Fix: deleted the stale S3 objects for those prefixes, deleted the corresponding stale table entries from the Glue Data Catalog, and re-ran the crawler — landing at exactly 18 tables (10 dimensions, 6 facts, the supplier bridge table, and api_orders).

13.3 Column-Naming Mismatch in Analytics SQL

Symptom: the originally drafted analytics queries (Tasks 29–41) joined on *_key columns (customer_key, product_key, date_key, etc.) and referenced dim_date.full_date — none of which exist in the actual tables.

Root cause: the dimension-building code aliases each source table's natural identifier directly (e.g. customerid → customer_id) rather than generating a separate surrogate key with a _key suffix, and dim_date's calendar column is named date. The SQL had been drafted against the naming the assignment template implied, not the naming the code actually produced.

Fix: rewrote every query in Section 12 to join on the real _id columns and dim_date.date. Also renamed the RANK() window-function alias from rank to product_rank / customer_rank, since RANK is a reserved keyword in Athena's underlying Trino/Presto engine and using it unquoted as a column alias raises a syntax error.

14. Known Limitations and Follow-ups

- dim_product.department_id is always NULL — no join path exists from products to departments in the source data.
- Dimension keys reuse the source system's natural ID rather than a generated surrogate key; acceptable for this project's scope, but would need revisiting if slowly-changing-dimension history is ever required.
- The Glue job recomputes all dimensions and facts from the full historical + full API JSON set on every run, rather than merging only newly arrived records — true minimal-reprocessing incrementality (Task 23) would need a processed-files manifest or Glue job bookmarks.
- dim_customer does not currently include a phone column; add it if the source customers.csv contains one and it's needed downstream.